

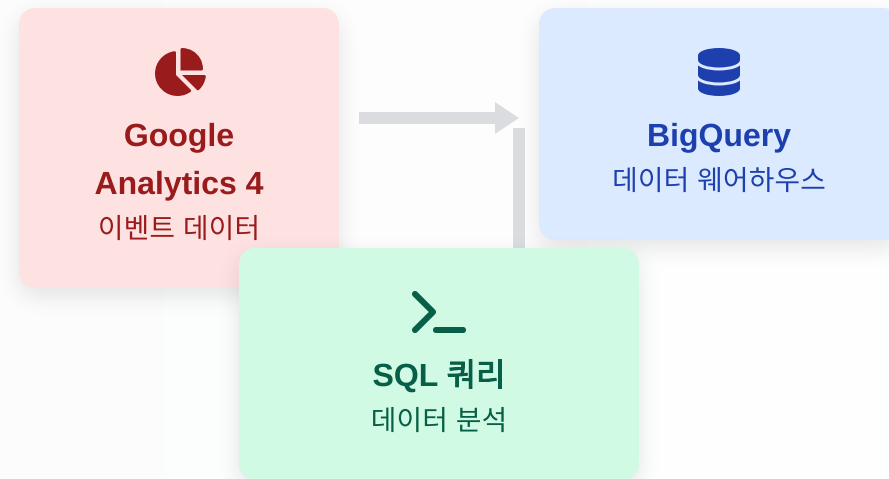
BigQuery란 무엇인가?

 BigQuery는 Google의 완전 관리형 **서버리스 데이터 웨어하우스**입니다.

 **대규모** 데이터 분석을 위해 설계되었습니다.

 대용량 데이터셋에 **빠른 SQL 쿼리**를 실행합니다.

 **Google Analytics 4 (GA4)**와 긴밀하게 통합되어 있습니다.



Google Analytics와 BigQuery를 함께 사용하는 이유



Google Analytics 4 (GA4)



샘플링된 데이터

GA4 UI에서는 샘플링되거나 요약된 데이터만 볼 수 있습니다.



기본 보고서만 제공

미리 정의된 보고서 및 제한된 분석 기능만 이용할 수 있습니다.



제한된 데이터 저장 기간

데이터 보존 기간의 제한이 있습니다.

VS



BigQuery



원시 데이터 (Raw Data)

샘플링되지 않은 모든 이벤트 수준 데이터에 직접 접근할 수 있습니다.



사용자 정의 분석

SQL을 통한 고급 분석 및 맞춤형 보고서를 생성할 수 있습니다.



다른 데이터와 통합 가능

CRM, 마케팅 데이터 등 다른 데이터 소스와 쉽게 결합할 수 있습니다.

SQL이란?

구조화 쿼리 언어 (Structured Query Language)

 **SQL**은 구조화된 데이터를 검색하고 조작하는 언어입니다.

```
SELECT * FROM users WHERE visit_date = '2024-05-30'
```

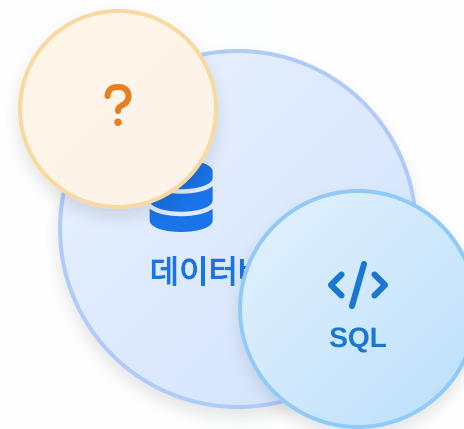
데이터베이스에 저장된 정보를 질의하고 분석하는 데 사용되며, BigQuery에서 GA4 데이터를 분석하는 핵심 도구입니다.

SQL로 할 수 있는 질문들:

 "어제 방문한 사용자는 몇 명인가요?"

 "전환율이 가장 높은 도시는 어디인가요?"

 "어떤 기기에서 가장 많은 주문이 발생하나요?"



SQL은 문장과 같습니다

SQL 구문과 일반 문장의 유사성

“ SQL 구문은 일반 문장과 매우 유사한 구조를 가집니다 ”

SQL 구문 구조:

SELECT → 보여줄 내용 (열)

FROM → 데이터 위치 (테이블)

WHERE → 조건 (필터)

일반 문장 구조:

[무엇을]

[어디서]

[어떤 조건으로]

"유저 ID와 방문 페이지를 사용자 로그에서 어제 방문한 경우만 보여줘"

SELECT

FROM

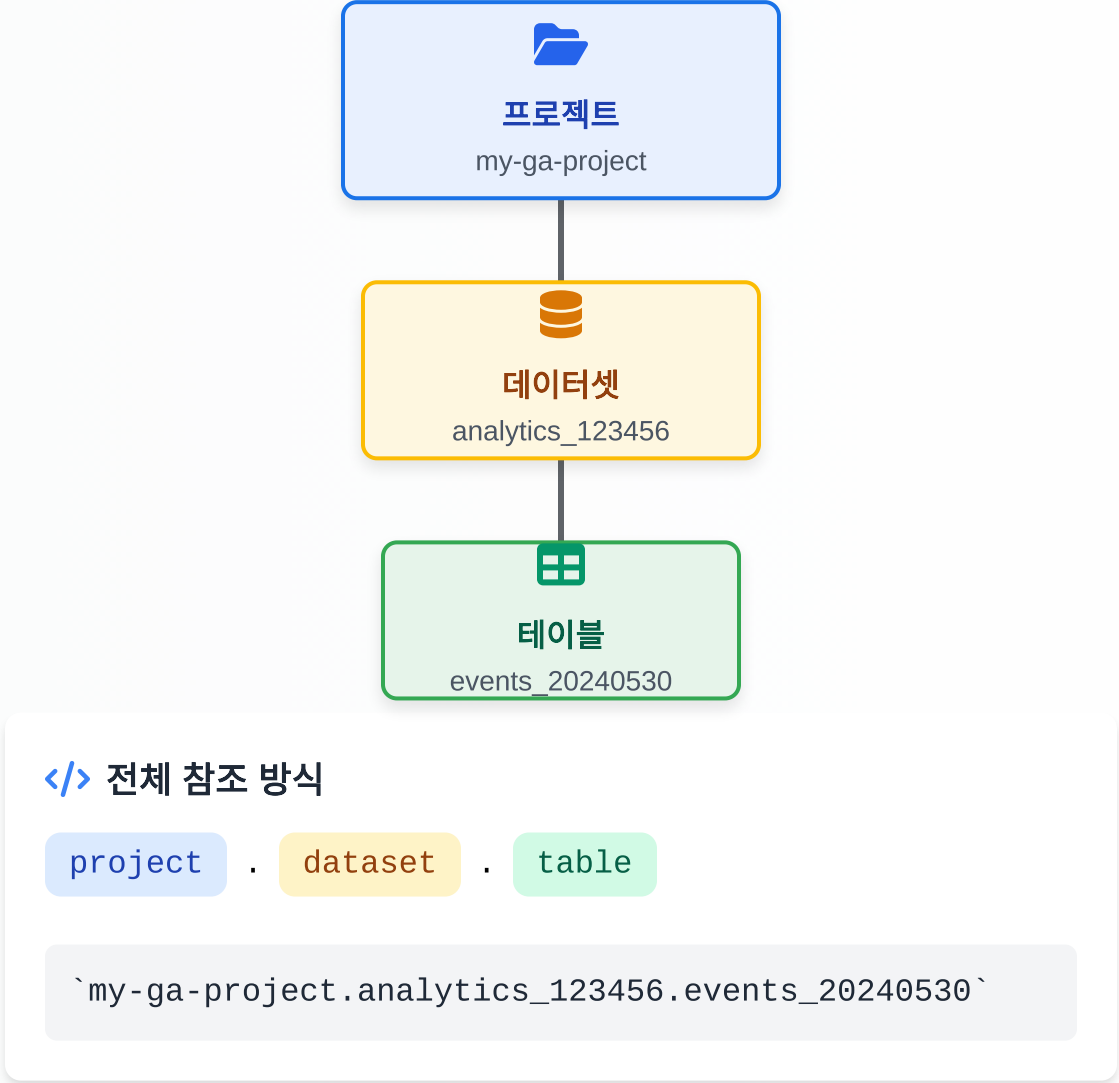
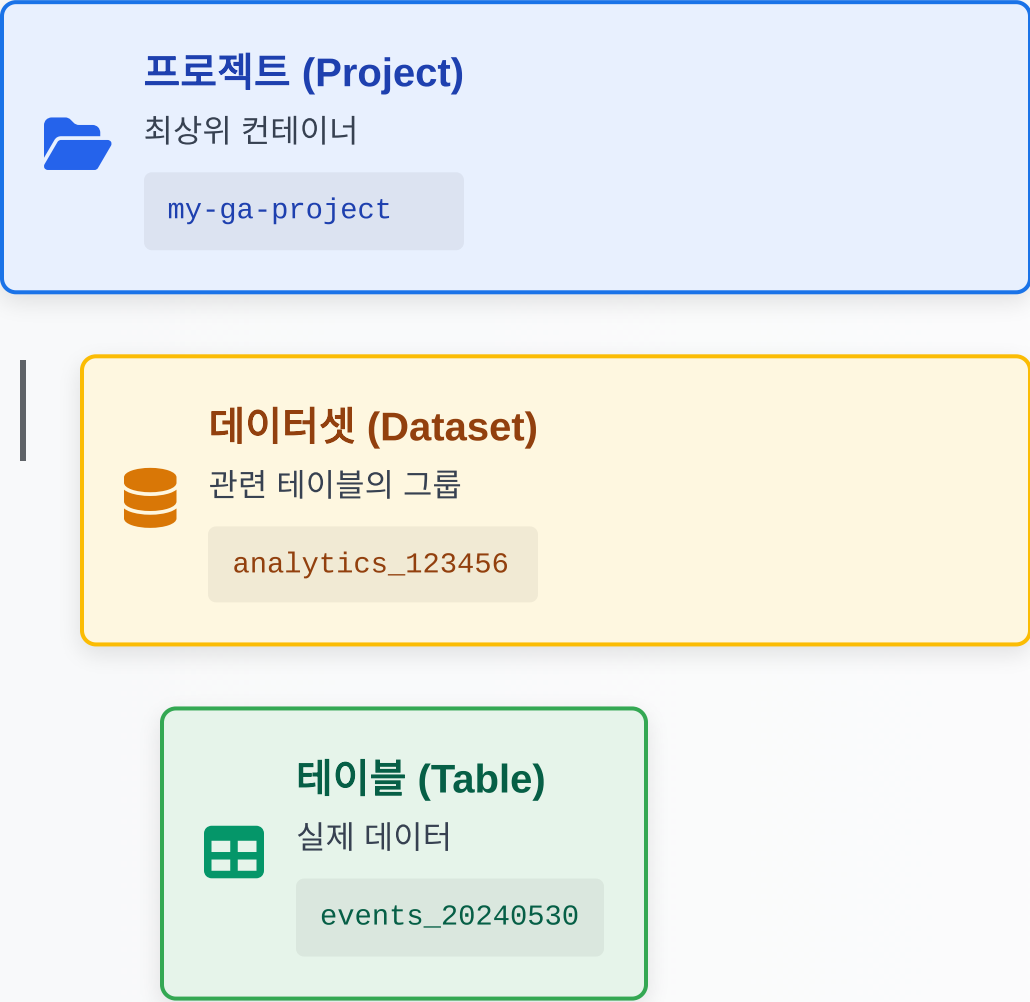
WHERE

```
SELECT user_id, page_name
FROM `project.dataset.events_20240530`
WHERE event_name = 'page_view';
```

💡 SQL은 "[이러한 열]을 [이 테이블]에서 [이런 조건]으로 보여줘"라고 요청하는 것과 같습니다.

BigQuery 구조

프로젝트, 데이터셋, 테이블 이해하기



Google Analytics 내보내기의 기본 테이블

GA4에서 BigQuery로 내보내지는 데이터 구조

일일 이벤트 테이블

Google Analytics 4는 매일의 이벤트 데이터를 별도의 테이블로 저장합니다:

`events_YYYYMMDD`

예시:

- `events_20240530` - 5월 30일 데이터
- `events_20240529` - 5월 29일 데이터
- `events_20240528` - 5월 28일 데이터

💡 실시간 분석을 위해서는 여러 일일 테이블을 조인해야 합니다.

테이블 스키마

GA4 이벤트 테이블은 다음과 같은 주요 필드를 포함합니다:

`event_name`

`user_pseudo_id`

`geo.city`

`device.category`

`event_params`

`event_timestamp`

`user_properties`

`traffic_source`



event_params와 **user_properties**는 중첩된 복잡한 구조를 가지며, 특별한 방법으로 쿼리해야 합니다.

데이터 계층 구조:

데이터 유형	필드 접근 방법
단순 필드	<code>event_name</code>
중첩 필드	<code>device.category</code>
배열/반복 필드	<code>UNNEST(event_params)</code>

</> 테이블 구조 예시

event_date	event_name	user_pseudo_id	geo.city	device.category
20240530	page_view	AB12345...	Seoul	mobile
20240530	click	AB12345...	Seoul	mobile
20240530	page_view	CD67890...	Busan	desktop

가장 기본적인 SQL 쿼리

SQL 쿼리 구조 이해하기

SQL Query

```
SELECT user_pseudo_id, event_name
FROM `project.dataset.events_20240530`
WHERE event_name = 'page_view';
```

SELECT

FROM

i 쿼리 실행 결과는?

이 쿼리는 **page_view** 이벤트를 발생시킨 모든 사용자의 ID와 이벤트명을 보여줍니다.

user_pseudo_id	event_name
A12BC34D...	page_view
E56FG78H...	page_view
I90JK12L...	page_view



SELECT

보여줄 열(columns) 지정

user_pseudo_id, event_name

→ 사용자 ID와 이벤트 이름을 보여줍니다.



FROM

데이터를 가져올 테이블 지정

`project.dataset.events\_20240530`

→ 2024년 5월 30일자 이벤트 테이블을 사용합니다.



WHERE

조건에 맞는 행(rows) 필터링

event_name = 'page_view'

→ 이벤트 이름이 'page_view'인 행만 선택합니다.



SQL은 문장처럼 읽을 수 있습니다

"사용자 ID와 이벤트 이름을 5월 30일 이벤트 테이블에서 가져와줘, 단 이벤트 이름이 'page_view'인 것만!"

테이블 이름 구조와 와일드카드

FROM 절 작성 방법

테이블 이름 구조

BigQuery 테이블 경로 형식:

```
`project.dataset.table`
```



최상위 컨테이너 프로젝트 관련 테이블 그룹 데이터셋 실제 데이터 테이블

실제 사용 예시

프로젝트와 데이터셋 구조

 프로젝트:
my-ga-project

 데이터셋:
analytics_123456

 테이블:
events_20240530

전체 테이블 경로:

```
`my-ga-project.analytics_123456.events_20240530`
```

와일드카드 사용하기

와일드카드란?
와일드카드(`*`)를 사용하면 여러 테이블을 한 번에 쿼리할 수 있습니다.

특히 유용한 경우: 여러 날짜의 GA4 일별 테이블을 함께 분석할 때

와일드카드 예시

여러 일자 데이터 쿼리하기

이 형식 대신:

```
`myproject.analytics_123456.events_20240530`,  
`myproject.analytics_123456.events_20240529`,  
`myproject.analytics_123456.events_20240528`
```

와일드카드 사용:

```
* `myproject.analytics_123456.events_*`
```

모든 날짜의 이벤트 테이블을 포함합니다

특정 기간 지정을 위한 필터:

```
SELECT event_name, COUNT(*) AS event_count  
FROM `myproject.analytics_123456.events_*`  
WHERE _TABLE_SUFFIX BETWEEN '20240501' AND  
'20240507'
```

_TABLE_SUFFIX: 와일드카드로 매칭된 테이블 이름의 `*` 부분을 의미합니다

중요!
BigQuery에서는 항상 백틱(```)으로 테이블 경로를 감싸야 합니다. 큰따옴표나 작은따옴표가 아닙니다.

값으로 필터링하기

목표: 모바일 사용자 찾기

SQL 쿼리

```
SELECT user_pseudo_id, device.category
FROM `project.dataset.events_*`
WHERE device.category = 'mobile';
```

이 쿼리가 하는 일

- 1 모든 이벤트 테이블에서 데이터를 가져옵니다.
- 2 `device.category`가 'mobile'인 레코드만 필터링합니다.
- 3 각 모바일 사용자의 ID와 기기 유형을 보여줍니다.

필터가 작동하는 방식



ID: A12BC34D...
device.category: desktop



ID: B34DE56F...
device.category: mobile



ID: C45FG67H

쿼리 결과 예시

user_pseudo_id	device.category
B34DE56F...	mobile
D78HI89J...	mobile
F90KL12M...	mobile

실무 팁



필터링은 데이터 분석의 기초입니다. **WHERE** 절을 사용하여 특정 조건에 맞는 데이터만 선별할 수 있습니다. 기기 유형뿐만 아니라 국가, 도시, 이벤트 유형 등 다양한 값으로 필터링이 가능합니다.

세션 또는 사용자별 그룹화하기

목표: 사용자별 이벤트 수 계산하기

SQL 쿼리

```
SELECT user_pseudo_id, COUNT(*) AS  
event_count  
FROM `project.dataset.events_*`  
GROUP BY user_pseudo_id;
```

이 쿼리가 하는 일

- 모든 이벤트 테이블에서 데이터를 가져옵니다.
- user_pseudo_id** 값이 같은 레코드끼리 그룹화합니다.
- 각 사용자 그룹별로 이벤트 수(**COUNT(*)**)를 계산합니다.

GROUP BY의 핵심 개념

GROUP BY는 지정된 열의 **고유한 값**을 기준으로 행을 그룹화합니다. 이를 통해 집계 함수(COUNT, SUM, AVG 등)를 각 그룹에 적용할 수 있습니다.

그룹화 작동 방식

원본 데이터:

A user_pseudo_id: ABC123
event_name: page_view

B user_pseudo_id: XYZ789
event_name: click

A user_pseudo_id: ABC123
event_name: scroll

A user_pseudo_id: ABC123
event_name: click

C user_pseudo_id: DEF456
event_name: page_view

GROUP BY

그룹화된 결과:

user_pseudo_id: ABC123

총 이벤트 수: 3

user_pseudo_id: XYZ789

총 이벤트 수: 1

user_pseudo_id: DEF456

총 이벤트 수: 1

쿼리 결과 예시

user_pseudo_id	event_count
ABC123	3
XYZ789	1
DEF456	1
GHI789	5

실무 팁



GROUP BY는 사용자 행동 패턴 분석에 필수적입니다. 사용자별, 세션별, 날짜별로 그룹화하여 트래픽 패턴이나 활동을 파악할 수 있습니다. 여러 열로 그룹화할 수도 있습니다. (예: GROUP BY user_pseudo_id, event_date)

날짜 범위로 필터링하기

목표: 특정 주간 데이터 분석하기

SQL 쿼리

```
SELECT event_date, COUNT(*) AS events
FROM `project.dataset.events_`
WHERE _TABLE_SUFFIX BETWEEN '20240501' AND '20240507'
GROUP BY event_date;
```

이 쿼리가 하는 일

1

_TABLE_SUFFIX 값을 기준으로 5월 1일~7일 데이터만 필터링합니다.

2

이벤트 날짜(event_date)로 데이터를 그룹화합니다.

3

각 날짜별 이벤트 발생 건수(COUNT)를 계산합니다.

테이블 접미사(_TABLE_SUFFIX) 활용

i

BigQuery에서 _TABLE_SUFFIX는 와일드카드(*)로 지정된 테이블의 뒷부분 이름을 나타냅니다.
이 예제에서는 GA4의 일별 테이블 이름(events_YYYYMMDD)에서 날짜 부분을 필터링합니다.

실무 팁

💡

날짜 필터링은 데이터 분석의 기본입니다. 일주일 단위, 한 달 단위로 데이터를 분석할 때 _TABLE_SUFFIX BETWEEN을 사용하면 효율적으로 필요한 날짜 범위만 조회할 수 있어 쿼리 성능이 향상됩니다. 특히 데이터가 많을수록 더 효과적입니다.

날짜 필터링 작동 방식

📅 BETWEEN '20240501' AND '20240507'

2024년 5월

일	월	화	수	목
28	29	30	1 ● events_20240501	2 ● events_20240502
5 ● events_20240505	6 ● events_20240506	7 ● events_20240507	8	9

2024년 5월 1일~7일 일별 이벤트 발생량

날짜	이벤트 수
5/1(수)	1,245
5/2(목)	1,378
5/3(금)	978
5/4(토)	842
5/5(일)	1,000
5/6(월)	1,450
5/7(화)	1,350

쿼리 결과 예시

event_date	events
20240501	1,245
20240502	1,378
20240503	978
20240504	842

이벤트 시간의 최소/최대값 구하기

목표: 이벤트 발생 시간 범위 찾기

SQL 쿼리

```
SELECT MIN(event_timestamp),  
MAX(event_timestamp)  
FROM `project.dataset.events_*`;
```

이 쿼리가 하는 일

- 모든 이벤트 테이블에서 가장 빠른 이벤트 시간을 찾습니다.
- 모든 이벤트 테이블에서 가장 늦은 이벤트 시간을 찾습니다.
- 두 값을 통해 전체 데이터 기간을 파악할 수 있습니다.

쿼리 결과 예시

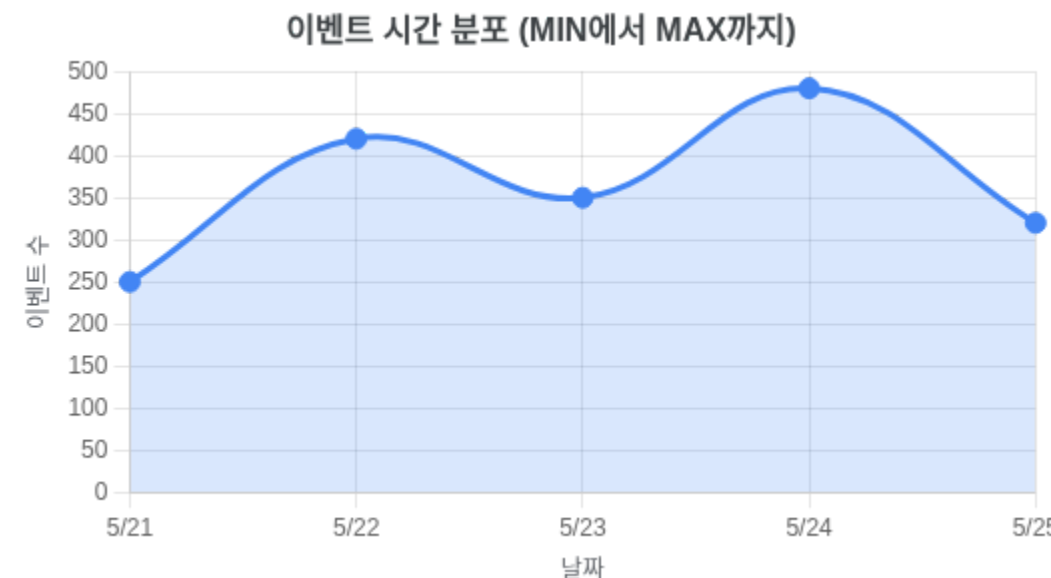
MIN(event_timestamp)	MAX(event_timestamp)
1716259200000000	1716604799000000

* event_timestamp는 마이크로초 단위의 유닉스 타임스탬프입니다.

💡 타임스탬프는 표준 날짜 함수로 변환할 수 있습니다:

```
SELECT  
  TIMESTAMP_MICROS(MIN(event_timestamp)) AS min_time,  
  TIMESTAMP_MICROS(MAX(event_timestamp)) AS max_time  
FROM `project.dataset.events_*`;
```

시간 범위 분석의 의미



집계 함수의 활용

MIN()과 MAX()는 SQL의 집계 함수로, 전체 데이터셋을 스캔하여 최소값과 최대값을 찾습니다. 이 외에도 다음과 같은 집계 함수를 활용할 수 있습니다:



- AVG() - 평균값 계산
- COUNT() - 행 수 계산
- SUM() - 합계 계산
- STDDEV() - 표준편차 계산

실무 팁

시간 범위 확인은 데이터 분석의 첫 단계입니다. 데이터가 어느 기간에 걸쳐 있는지 파악하면 다음과 같은 이점이 있습니다:



- 데이터 품질 문제 발견 (예: 누락된 날짜)
- 적절한 분석 기간 선택
- 이상치 탐지 (예: 미래 날짜의 데이터)
- 분석 결과의 타임라인 설정

도시별 필터링하기

목표: 서울에서 발생한 이벤트만 표시하기

SQL 쿼리

```
SELECT user_pseudo_id, geo.city
FROM `project.dataset.events_*`
WHERE geo.city = 'Seoul';
```

이 쿼리가 하는 일

- 1 모든 이벤트 테이블에서 사용자 ID와 도시 정보를 선택합니다.
- 2 중첩된 필드 **geo.city**를 사용해 지리적 위치를 필터링합니다.
- 3 'Seoul'이라는 값과 정확히 일치하는 레코드만 반환합니다.

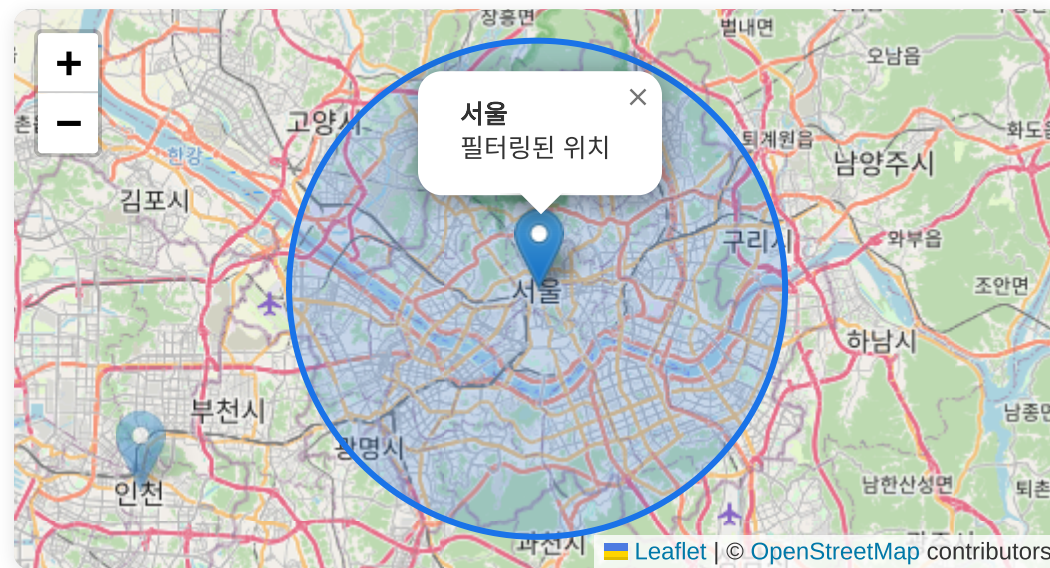
지리 데이터 활용 팁

GA4는 다양한 지리 정보를 제공합니다:

- **geo.country** - 국가
- **geo.region** - 지역/주
- **geo.city** - 도시
- **geo.metro** - 대도시 지역(미국)

이러한 필드를 활용하여 지역별 사용자 행동을 분석할 수 있습니다.

📍 지리적 필터링 시각화



데이터 필터링 과정:

부산 인천 **서울** 대구 광주 대전 울산 수원 제주

부산 인천 **서울** 대구 광주 대전 울산 수원 제주

📊 쿼리 결과 예시

user_pseudo_id	geo.city
1528374659284735	📍 Seoul
2639485726394857	📍 Seoul
3749582736485927	📍 Seoul
4857293847561928	📍 Seoul

실무 팁



지리적 필터링은 지역별 마케팅 효과 측정이나 지역 타겟팅 캠페인 분석에 유용합니다. 도시 이름은 대소문자를 구분하므로 정확한 표기를 사용해야 합니다. 여러 도시를 한 번에 필터링하려면 `WHERE geo.city IN ('Seoul', 'Busan', 'Incheon')`와 같이 `IN` 연산자를 사용할 수 있습니다.

또한 도시명이 정확하게 일치하지 않을 수 있으므로, `WHERE geo.city LIKE '%Seoul%'`와 같이 `LIKE` 연산자를 사용해 부분 일치를 찾는 것도 좋은 방법입니다.

이벤트 참여 사용자 찾기

목표: 특정 이벤트 요소를 클릭한 사용자 식별하기

고급 SQL 쿼리

```
SELECT
  user_pseudo_id,
  geo.city,
  device.category,
  device.mobile_model_name,
  device.operating_system,
  MIN(user_first_touch_timestamp) AS
first_visit_ts
FROM `project.dataset.events_*`
WHERE
  event_name = '이벤트참여'
  AND EXISTS (
```

복잡한 쿼리 분석

- 기본 필드 선택:** 사용자 ID, 도시, 기기 정보 등을 선택합니다.
첫 방문 시간도 함께 추출(MIN 집계 함수 사용)
- 이벤트 필터링:** '이벤트참여' 이름을 가진 이벤트만 대상으로 합니다.
- 중첩 데이터 처리:** UNNEST를 사용해 event_params 배열을 펼칩니다.
특정 CSS 선택자와 일치하는 요소에 대한 이벤트만 추출
- GROUP BY:** 사용자별로 데이터를 그룹화합니다.
여러 이벤트 참여가 있더라도 사용자당 하나의 행만 반환

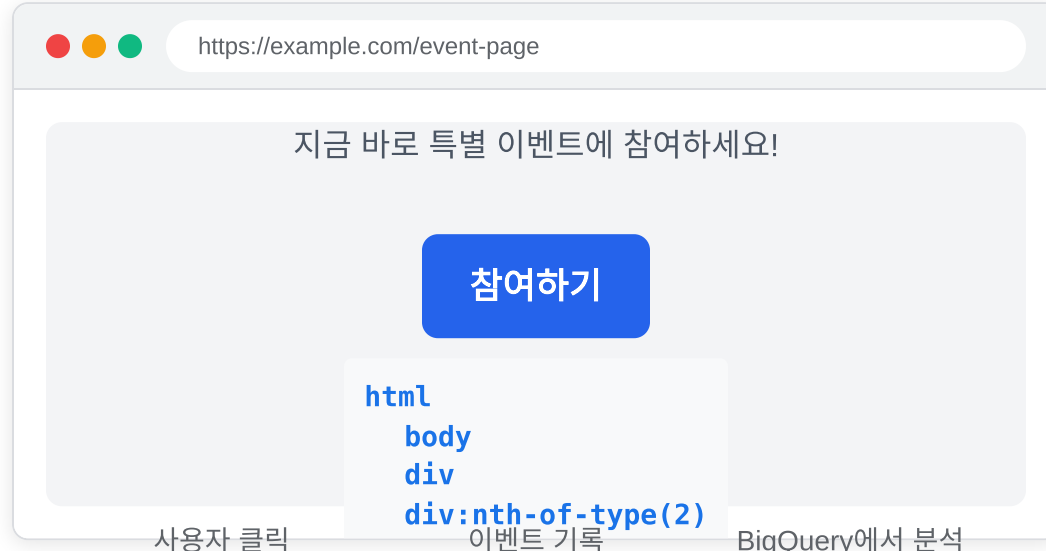
핵심 포인트



복잡한 이벤트 추적이 필요할 때는 **CSS 선택자**를 활용하여 정확한 사용자 인터랙션을 분석할 수 있습니다. 이 방법은 웹사이트 내 특정 영역이나 요소와의 상호작용을 정밀하게 추적하고자 할 때 매우 유용합니다.

UNNEST 함수를 사용하면 GA4의 중첩된 데이터 구조(event_params와 같은 배열)를 효과적으로 처리할 수 있습니다.

이벤트 요소 시각화



쿼리 결과 활용 사례

이벤트 참여자 데이터 샘플

기기 종류	운영체제	도시
mobile	iOS 16.5	서울
mobile	Android 13	부산
desktop	Windows 11	인천

활용 사례

- 특정 버튼 클릭자에게 **맞춤형 리마케팅** 캠페인 진행
- 지역별 이벤트 참여율 분석으로 **지역 타겟팅** 전략 수립
- 기기 유형별 이벤트 참여 패턴 파악으로 **UX 최적화**
- 이벤트 참여자 **코호트 분석**으로 장기적 행동 패턴 추적

AI 도구인 Gemini를 BigQuery에 활용하는 이유

실용적인 도우미, 대체제가 아닙니다

Gemini가 SQL 작업에 도움이 되는 방법



SQL 쿼리 작성 및 설명

Gemini(또는 ChatGPT 등)는 SQL 쿼리를 작성하고 설명할 수 있습니다.



시간 절약

구문이나 논리가 불확실할 때 도움을 받아 시간을 절약할 수 있습니다.



명확한 질문의 중요성

명확하게 질문해야 합니다 — 좋은 프롬프트가 유용한 결과로 이어집니다.

기억하세요



AI는 대체제가 아닌 도구입니다. SQL과 데이터 구조에 대한 기본적인 이해는 여전히 필요합니다.

AI의 제안을 그대로 수용하기보다는 검증하고 이해한 후 활용하세요.

Gemini로 SQL 쿼리 작성하기

사용자

GA4의 이벤트 데이터에서 지난 7일간 모바일 기기를 사용한 방문자 수를 BigQuery로 어떻게 조회할 수 있을까요?



Gemini

네, 다음 SQL 쿼리를 사용하여 GA4의 지난 7일간 모바일 기기 방문자 수를 조회할 수 있습니다:

```
SELECT
  COUNT(DISTINCT user_pseudo_id) AS
  mobile_users
FROM `project.dataset.events_*`
WHERE
  _TABLE_SUFFIX BETWEEN
  FORMAT_DATE('%Y%m%d',
    DATE_SUB(CURRENT_DATE(), INTERVAL 7
    DAY))
  AND FORMAT_DATE('%Y%m%d',
    CURRENT_DATE())
  AND device.category = 'mobile';
```

이 쿼리는:

- 지난 7일 동안의 이벤트 테이블을 대상으로 합니다
- 모바일 기기만 필터링합니다
- 중복 사용자를 제거하고 고유 방문자 수를 계산합니다

사용자

감사합니다! 이 쿼리에 도시별로 그룹화하는 기능도 추가할 수 있을까요?

AI 활용의 장점



빠른 프로토타이핑

복잡한 쿼리를 빠르게 생성



학습 도구

SQL 작성법과 최적화 학습



디버깅 지원

오류 해결 및 코드 개선

SQL을 위한 AI 프롬프팅의 3가지 황금 규칙

효과적인 SQL 쿼리 생성을 위한 AI 프롬프트 작성법



1

구체적으로 작성하기

원하는 정보의 세부 사항을 명확하게 명시하세요.

- ✓ 어떤 **컬럼**이 필요한지
- ✓ 어떤 **조건**으로 필터링할지
- ✓ 어떤 **기간**의 데이터가 필요한지

예시:

~~"사용자 분석해줘"~~

"지난 7일간 모바일 기기 접속 사용자의 도시와 방문 횟수"



2

GA4 구조 언급하기

GA4의 데이터 구조를 프롬프트에 포함하세요.

- ✓ 테이블 형식: events_YYYYMMDD
- ✓ 중첩 필드: event_params
- ✓ 사용자 식별자: user_pseudo_id

예시:

~~"GA4 데이터에서 클릭 찾아줘"~~

"GA4 BigQuery events_* 테이블에서 event_name이 'click'인 이벤트와 그 event_params 분석"



3

목표 기반으로 설명하기

무엇을 '하고 싶은지'가 아닌, 무엇을 '보고 싶은지' 설명하세요.

- ✓ 원하는 **인사이트**를 명확히
- ✓ 분석의 **비즈니스 목적** 설명
- ✓ 데이터로 **답하려는 질문** 포함

예시:

~~"SQL 쿼리 작성해줘"~~

"어느 지역과 디바이스에서 구매 전환율이 가장 높은지 보여주는 쿼리"

핵심 포인트



명확한 프롬프트 = 정확한 결과

이 세 가지 규칙을 따르면 AI가 정확하고 유용한 SQL 쿼리를 생성할 가능성이 크게 높아집니다. 어떤 결과가 필요한지를 최대한 자세히 설명하고, GA4의 데이터 구조에 관한 정보를 제공하며, 원하는 인사이트를 명확하게 전달하세요.

사용할 수 있는 프롬프트 공식

Gemini를 위한 프롬프트 템플릿



BigQuery SQL 프롬프트 템플릿

"Write a BigQuery SQL query for GA4 data exported to BigQuery.

I want to see [metrics/columns]
for users who [condition/action]
from [time period or table],
and optionally [grouping or aggregation]."

템플릿 분석

[metrics/columns] → 보고 싶은 정확한 데이터 필드

[condition/action] → 사용자 행동 또는 필터링 조건

[time period] → 날짜 범위 또는 테이블 지정

[grouping] → 그룹화 방식 및 집계 함수

중요: GA4와 BigQuery를 명시적으로 언급하세요. 이렇게 하면 AI가 올바른 데이터 구조와 함수를 사용합니다.



프롬프트 예시와 응용

프롬프트 예시

"Write a query that shows user ID, device model, and city
for users who clicked an element with selector
' .btn-promo '
between May 1 and May 7, 2024."

좋은 이유: 구체적인 필드, 명확한 조건, 정확한 날짜 범위를 명시

다양한 응용 방법

전환율 분석

"Write a BigQuery SQL for GA4 that calculates conversion rates by source and device for completed purchases in the last 14 days."

사용자 행동 분석

"Create a query to find users who abandoned their cart (added items but didn't purchase) and their last viewed page in March 2024."

이벤트 세션 분석

"Write BigQuery SQL to analyze event sequences within sessions showing how many users navigated from homepage to product page to checkout."



프로팁: 너무 복잡한 분석은 여러 단계로 나누세요. 먼저 기본 데이터를 가져온 후, 그 결과를 기반으로 추가 분석을 요청하세요.

프롬프트 마스터 되기



템플릿을 시작점으로 사용하고, 점차 자신만의 프롬프트 패턴을 개발하세요. AI의 응답을 분석하고 프롬프트를 개선하는 과정을 반복할수록 더 나은 결과를 얻을 수 있습니다.

🗨️ 명확한 질문

🔄 반복적 개선

📊 구조 활용

예시 프롬프트 → 응답 분석

AI 프롬프트와 생성된 SQL 쿼리의 비교 분석

프롬프트



BigQuery SQL을 작성해서 다음을 찾아주세요:
2024년 4월 동안 'purchase' 이벤트를 트리거한 사용자의

- 디바이스 모델
- 운영체제
- 접속 도시

핵심 요소 분석

- ✓ 데이터 대상: BigQuery SQL, GA4
- ✓ 원하는 데이터: 구체적인 필드 명시
- ✓ 조건: 특정 이벤트('purchase')
- ✓ 기간: 명확한 시간 범위(2024년 4월)

Gemini 응답



다음은 2024년 4월 동안 구매 이벤트를 발생시킨 사용자의 디바이스 모델, 운영체제, 도시를 보여주는 BigQuery SQL 쿼리입니다:

```
SELECT
  user_pseudo_id,
  device.mobile_model_name,
  device.operating_system,
  geo.city
FROM `set.events_*`
WHERE
  event_name = 'purchase'
  AND _TABLE_SUFFIX BETWEEN '20240401' AND '20240430';
```

요청한 필드 정확히 반영

테이블 접미사로 날짜 필터링

쿼리 설명: 이 쿼리는 GA4에서 BigQuery로 내보낸 이벤트 테이블에서 'purchase' 이벤트를 찾습니다. 테이블 접미사를 사용하여 4월 1일부터 30일까지의 데이터만 조회합니다.

분석 결과



명확한 프롬프트는 정확한 쿼리를 생성합니다. 위 예제에서 볼 수 있듯이, AI는 요청된 필드(디바이스 모델, OS, 도시)를 정확히 선택하고, 특정 이벤트('purchase')로 필터링하며, 테이블 접미사를 사용하여 올바른 날짜 범위(2024년 4월)를 설정했습니다. 사용자의 명확한 지시가 있으면 AI는 복잡한 SQL 문법을 올바르게 구성할 수 있습니다.

AI 프롬프팅의 일반적인 실수

명확한 프롬프트와 불명확한 프롬프트 비교

⚠️ 흔한 실수 사례

불명확

"분석 데이터 보여줘"



명확

"2024년 5월 3일 홈페이지 방문자의 도시별 세션 수 보여줘"

불명확

"GA4 쿼리"



명확

"GA4에서 BigQuery로 내보낸 테이블을 위한 SQL 쿼리"

불명확

"페이지 조회수 알려줘"



명확

"events_* 테이블에서 2024년 2분기 동안 page_view 이벤트 수를 일별로 계산해줘"

💡 효과적인 프롬프팅 가이드라인



"사용자 정보 가져와"

목적, 필드, 조건이 없음



"지난 주 구매한 사용자의 디바이스 유형과 도시 정보 가져와"

구체적인 기간, 이벤트, 필드 명시



"page_view에 관한 모든 정보 보여줘"

너무 광범위함



"page_view 이벤트의 상위 10개 페이지 경로와 각 조회수 보여줘"

구체적인 분석 목표와 결과물 명시

★ AI 프롬프팅 팁

- > 가능하면 항상 **이벤트 이름**이나 **GA4 필드명**을 언급하세요.
- > 데이터를 필터링할 **날짜 범위**를 명시하세요.
- > 어떤 **비즈니스 질문**에 답하려는지 설명하세요.
- > **테이블 구조**(events_*, events_intraday_)를 언급하세요.

“

"좋은 프롬프트는 분석가의 의도를 명확히 전달하고, 데이터 구조에 대한 이해를 반영합니다. 불확실한 부분이 있으면, 그 불확실성을 프롬프트 자체에 명시하는 것도 좋은 방법입니다."

핵심: 항상 이벤트 이름, 필드명, 날짜 범위, 그리고 원하는 결과물의 형태를 구체적으로 명시하세요.

연습: 아래 프롬프트를 더 좋게 개선해보세요

불명확한 프롬프트를 구체적인 프롬프트로 바꿔보세요

✕ "사용자 정보 가져와"

! 어떤 사용자? 어떤 정보? 어느 기간? 어떤 조건?

개선 필요

✕ "page_view에 대한 모든 것 보여줘"

! 너무 광범위함. 어떤 측면? 어떤 기간? 어떤 세그먼트?

개선 필요

✕ "클릭 데이터 찾아줘"

! 어떤 요소의 클릭? 어느 페이지에서? 무슨 기간 동안?

개선 필요

🔧 다음 요소를 포함하도록 개선하세요

- ✓ 원하는 열(필드) 구체적으로 명시
- ✓ 이벤트 이름 또는 관련 GA4 매개변수
- ✓ 조회할 날짜 또는 기간
- ✓ 필터링할 조건 (어떤 사용자, 어떤 행동)

💡 예시: 개선된 프롬프트

"지난달 프로모션 페이지를 방문한 모바일 사용자들의 사용자 ID, 도시, 기기 모델을 볼 수 있는 BigQuery SQL을 작성해줘."

→ 구체적인 기간, 조건, 원하는 필드, 타겟 테이블이 모두 포함됨

✍️ 작업 지시사항

위의 세 가지 불명확한 프롬프트를 개선해서 다시 작성해 보세요.

각각에 열(필드), 이벤트 이름, 날짜, 조건을 추가하여 명확하고 구체적인 프롬프트로 만들어야 합니다.

힌트: 실제 비즈니스 상황을 상상해 보세요. 마케팅 담당자라면 어떤 데이터를 필요로 할까요?

AI의 답변에서 배우기

단순히 사용만 하지 말고, 이해하고 배우세요

```
SELECT
  user_pseudo_id,
  device.mobile_model_name,
  device.operating_system,
  geo.city
FROM `project.dataset.events_*`
WHERE
  event_name = 'purchase'
  AND _TABLE_SUFFIX BETWEEN '20240401' AND '20240430'
  AND EXISTS (
    SELECT 1 FROM UNNEST(event_params) AS param
    WHERE param.key = 'promotion_id'
    AND param.value.string_value = 'summer_sale'
  )
GROUP BY
  1, 2, 3, 4
HAVING
  COUNT(*) > 1
```

💡 AI가 생성한 쿼리는 학습 자료입니다

단순히 복사하여 실행하지 말고, 각 부분이 무엇을 하는지 분석하고 이해하세요. 이런 학습 과정이 여러분의 SQL 역량을 빠르게 향상시킵니다.

🎓 AI의 쿼리에서 배울 점



왜 이 테이블을 사용했나요?

AI가 선택한 테이블과 테이블 구조(events_*, events_intraday_ 등)가 왜 적절한지 이해하세요. 다른 테이블은 어떤 상황에 사용할까요?



각 절(clause)의 의미는 무엇인가요?

WHERE, GROUP BY, HAVING, ORDER BY 등의 절이 어떤 역할을 하는지 분석하세요. 왜 이런 순서로 작성되었나요?



어떻게 수정하면 다른 용도로 활용할 수 있을까요?

생성된 쿼리를 기반으로 다른 조건, 기간, 이벤트에 맞게 수정해보세요. 이 과정에서 SQL 활용 능력이 향상됩니다.



AI에게 질문하세요

"이 쿼리의 EXISTS 구문은 왜 사용했나요?" 또는 "UNNEST 함수의 역할이 무엇인가요?" 와 같은 후속 질문으로 더 깊은 이해를 얻으세요.

핸즈온 활동

Gemini에게 물어보고, 직접 수정해보세요

? 비즈니스 질문 선택하기



이벤트 프로모션 참여자는 누구인가?

특정 이벤트를 클릭하거나 참여한 사용자의 정보 분석하기

💡 관련 이벤트: `event_name = '이벤트참여'`



모바일에서 방문한 사용자 수는?

기기 유형별 방문자 수 및 특성 분석하기

💡 관련 필드: `device.category = 'mobile'`

📋 Gemini 프롬프트 작성 단계

1. 위 비즈니스 질문 중 하나를 선택하세요
2. 구체적인 프롬프트 템플릿을 사용하세요
3. 원하는 필드, 기간, 조건을 명시하세요
4. 쿼리 결과의 정렬, 제한도 요청하세요

</> 쿼리 수정하기

생성된 쿼리

Gemini 응답

```
SELECT
  user_pseudo_id,
  device.category,
  device.mobile_model_name,
  geo.city,
  COUNT(*) AS event_count
FROM `project.dataset.events_*`
WHERE
  event_name = '이벤트참여'
  AND _TABLE_SUFFIX BETWEEN '20240501' AND '20240507'
GROUP BY
  user_pseudo_id,
  device.category,
  device.mobile_model_name,
  geo.city
```



WHERE 조건 수정하기 직접 수정

특정 이벤트 파라미터나 특성으로 필터링 추가하기

💡 `AND EXISTS (SELECT 1 FROM UNNEST(event_params) ...)`
구문 활용해보기



날짜 범위 변경하기 직접 수정

다른 기간으로 변경하여 트렌드 확인하기

💡 `_TABLE_SUFFIX BETWEEN 'YYYYMMDD' AND 'YYYYMMDD'` 수정하기



열(컬럼) 추가/변경하기 직접 수정

추가 정보나 집계 함수 포함하여 분석 강화하기

💡 `MIN()`, `MAX()`, `AVG()` 같은 집계 함수나 `EXTRACT()` 활용하기

AI + 분석가 = 강력한 조합

각자의 강점을 활용하여 함께 성장하는 방법



AI의 역할

- 복잡한 SQL 빠르게 생성
- 구문 오류 최소화
- 다양한 패턴 제안
- 초보자도 빠른 시작



분석가의 역할

- 비즈니스 맥락 이해
- 올바른 질문 설정
- 데이터 품질 판단
- 통찰력 있는 해석



시너지 효과

- 실행 속도 향상
- 더 다양한 분석 시도
- 기술적 한계 극복
- 지속적인 학습 가속화

! 여전히 필요한 역량

- 어떤 데이터가 있는지 이해하기
- 올바른 비즈니스 질문을 설정하기
- 결과를 적절히 해석하는 능력
- 쿼리 결과의 타당성 검증하기
- 다양한 데이터 소스 연결하기

👋 AI의 한계 인지하기

AI는 여러분의 비즈니스 맥락을 온전히 이해하지 못합니다. 생성된 쿼리는 항상 검토하고 수정해야 합니다.

📈 AI와 함께 성장하는 전략



생성, 학습, 반복

AI로 초안을 만들고, 분석하며 배우고, 개선하세요



더 나은 질문하기

AI의 답변을 보고 더 깊은 후속 질문을 생성하세요



패턴 라이브러리 구축

자주 쓰는 쿼리 패턴을 수집하고 응용하세요



팁: AI와 협업은 기술이 아닌 마인드셋입니다. AI를 활용하면서도 스스로 발전하는 균형을 찾으세요.

GA4 BigQuery 데이터셋의 기본 테이블

GA4가 BigQuery와 연결될 때 제공되는 테이블



events_ 가장 많이 사용

일별로 내보낸 이벤트 테이블 (형식: events_YYYYMMDD)

포함 데이터:

- ✓ 페이지뷰, 클릭, 전환 등 사용자 행동
- ✓ event_name, user_pseudo_id, geo.city 등
- ✓ device.category, 이벤트 매개변수 등

활용 방법:

```
</> 여러 날짜를 쿼리하려면: FROM `project.dataset.events_*`  
</> 특정 날짜만: FROM `project.dataset.events_20240530`
```



events_intraday_ 실시간에 가까움

당일 이벤트를 위한 준실시간 데이터 (몇 시간마다 업데이트)

특징:

- ✓ 몇 시간마다 업데이트됨
- ✓ events_ 테이블과 동일한 스키마
- ✓ 당일 대시보드에 적합

활용 예:

```
-- 가장 최신 데이터 통합하여 보기  
SELECT * FROM ( SELECT * FROM  
`project.dataset.events_*` UNION ALL SELECT * FROM  
`project.dataset.events_intraday_*` )
```



pseudonymous_users 선택 활성화

사용자 수준 메타데이터 (사용자 내보내기가 활성화된 경우)

제공 정보:

- ✓ 사용자 생애 주기 통계
- ✓ 첫 방문 시간, 마지막 방문 등
- ✓ events_ 테이블과 조인하여 사용

조인 예:

```
SELECT e.event_name, u.first_touch_date FROM  
`project.dataset.events_*` e JOIN  
`project.dataset.pseudonymous_users` u ON  
e.user_pseudo_id = u.user_pseudo_id
```

테이블 간 관계:


events_YYYYMMDD
과거 데이터

+


events_intraday_
오늘의 데이터

=


전체 분석 데이터
과거+현재 통합

 events_* 를 사용하면 모든 날짜의 데이터를 한 번에 쿼리할 수 있습니다